

VOLUMEN V · LIBRO 5 DE 5

BMAD Protocolo TEA

El protocolo de calidad: marcos mentales, skills y escenarios reales

Cuatro partes: marcos mentales, skills en uso,
escenarios cross-cutting y recursos.
El libro al que vuelves cuando algo no cuadra.

Test Architect · BMAD Method

2026-05-07 · LIDR.co

0 Bienvenida

Este es el quinto y último libro de la biblioteca. Es la referencia técnica del módulo Test Architect (TEA): los marcos mentales que estructuran las decisiones de testing, las nueve skills que los aplican, los escenarios reales en los que se usan y los recursos a los que volver cuando algo no cuadra.

A diferencia de los anteriores, este libro tiene un sesgo claro hacia perfiles técnicos — QA, Tech Lead, Architect — aunque el Sec. 2 (Por qué TEA) y los Sec. 18–Sec. 21 (escenarios cross-cutting) son hojeables por cualquier rol del equipo.

0.1 Cómo está organizado el libro

El libro está dividido en cuatro partes. El orden importa: cada parte construye sobre la anterior.

Parte	Contenido	Para qué sirve	Quién la lee
A — Marcos mentales	Sec. 2–Sec. 7. Seis marcos transversales que TEA aplica.	Establece el vocabulario y el modelo mental antes de tocar ninguna skill.	Todos los roles que vayan a interactuar con TEA.
B — Skills	Sec. 8–Sec. 17. Mapa rápido y nueve fichas, una por skill.	Referencia operativa para cuando se va a invocar una skill concreta.	QA y Tech Lead principalmente.
C — Escenarios	Sec. 18–Sec. 21. Cuatro patrones de uso reales con runbook paso a paso.	Aplicación de los marcos y las skills en situaciones que ocurren en proyectos reales.	Tech Lead, Scrum Master, oncall.
D — Recursos	Sec. 22–Sec. 25. Persona Murat, listado del knowledge base, plantillas, verificación.	Para profundizar en patrones específicos y disponer de plantillas listas.	Como referencia, no como lectura lineal.

0.2 Cuándo abrir este libro

Situación	Sección que toca
Onboarding al rol QA o Test Architect	PARTE A entera, después PARTE B mapa (Sec. 8) como índice de skills.
Vas a ejecutar una skill TEA	La ficha correspondiente en PARTE B (Sec. 9–Sec. 17).
Hay un incidente en producción	Sec. 18 — Hotfix. Runbook con cronograma.
Tests inestables sospechados	Sec. 19 — Cuarentena de tests flaky.
Equipo nuevo con suite legacy	Sec. 20 — Auditoría brownfield (4 etapas).
Algo no funciona como esperabas	Sec. 21 — Diagnóstico rápido de pitfalls comunes.
Necesitas plantilla para presentar a stakeholders	Sec. 24 — Plantillas (Risk Matrix, NFR Criteria, Gate Report, ADR Quality Readiness Checklist).

1 Convenciones

1.1 Insignias y colores

● **TEA** · testing ● **BMM** · flujo principal (cuando aparece)

REQ TEA obligatoria si el equipo adopta TEA · **REC** recomendada · **OPC** opcional.

VISIÓN LIDR framing nuestro · **RECOMENDACIÓN LIDR** sugerencia operativa.

Para los niveles de prioridad de tests:

P0 Critical · **P1** High · **P2** Medium · **P3** Low.

1.2 Anatomía de las tarjetas

Tres tipos de tarjeta aparecen en el libro:

- **Marco** — cuadro verde con etiqueta "Marco" para los conceptos transversales de la PARTE A.
 - **Skill** — tarjeta con cabecera y siete bloques (resuelve, cuándo, prerequisites, salida, sub-pasos, duración, ver también) idéntica al Libro 4.
 - **Escenario** — cuadro naranja con etiqueta "Escenario" para los patrones de uso de la PARTE C.
-

PARTE A

Marco mental — el porqué antes que el cómo

Seis marcos transversales que TEA aplica a cualquier proyecto. Si solo lees una parte de este libro, lee esta. Las skills (Parte B) son la herramienta operativa; los marcos son el lenguaje compartido del equipo.

2 Por qué TEA

Antes de hablar de skills, marcos o decisiones, conviene fijar qué problema concreto resuelve el módulo y qué se gana al adoptarlo.

2.1 El problema

En un equipo que escribe tests sin protocolo común aparecen, con altísima frecuencia, los mismos cuatro síntomas:

- Suite que crece pero no protege: muchos tests, baja confianza al desplegar.
- Tests inestables que el equipo aprende a ignorar (o peor, a re-ejecutar hasta que pasen).
- Cobertura medida en porcentaje, sin relación con el riesgo real del producto.
- Ausencia de criterios objetivos para decidir si una release está lista o no.

2.2 Lo que aporta TEA

El módulo introduce cuatro disciplinas que se sostienen entre sí:

1. **Riesgo basado en negocio** — cada test tiene una prioridad P0–P3 derivada de probabilidad y de impacto, no de cuándo se ejecuta en CI (Sec. 3).
2. **Niveles correctos** — cada comportamiento se prueba en el nivel más barato que lo cubra: unit antes que integration, integration antes que end-to-end (Sec. 4).
3. **Calidad de tests medible** — ocho criterios concretos que un test debe cumplir; los tests que no los cumplen son deuda, no activo (Sec. 5).
4. **Decisiones de gate explícitas** — antes de cada release, una decisión documentada (PASS, CONCERNS, WAIVED, FAIL) basada en evidencia, no en sensaciones (Sec. 6).

Sobre estas cuatro disciplinas se asientan dos invariantes operativas: la **traceability viva** entre requisitos y tests (Sec. 7) y la **regla bloqueante de orden** entre las skills TEA (Sec. 8.2).

2.3 A qué se renuncia

Adoptar TEA no es gratis. El equipo asume tres compromisos:

- Aceptar que *no todo se prueba al mismo nivel* — esto requiere conversaciones que no ocurren en suites sin protocolo.
- Mantener la traceability story-by-story, no de manera retroactiva.
- Bloquear releases con CONCERNS o FAIL aunque el equipo de producto presione para liberar.

Si alguno de los tres compromisos no es asumible en el contexto del proyecto, conviene posponer la adopción hasta que lo sea.

3 Riesgo basado en negocio

Punto de partida del módulo. Sin clasificación de riesgo, la suite trata todos los tests como si fueran igual de importantes; eso resulta en suites grandes y caras que no protegen lo que importa.

MARCO

Probability × Impact → score → prioridad

Cada riesgo se evalúa con dos escalas independientes de tres niveles cada una.

3.1 ESCALA DE PROBABILIDAD

Nivel	Etiqueta	Cuándo aplica
1	Improbable	Implementación estándar, baja incertidumbre, patrones conocidos.
2	Posible	Casos de borde o desconocidos parciales; patrón conocido pero con variantes.
3	Probable	Problemas conocidos, integraciones nuevas, alta ambigüedad de requisitos.

3.2 ESCALA DE IMPACTO

Nivel	Etiqueta	Qué causa si ocurre
1	Menor	Problema cosmético o con workaround sencillo del usuario.
2	Degradado	Pérdida parcial de funcionalidad o requiere workaround manual.
3	Crítico	Bloqueo total; exposición de datos, seguridad o cumplimiento.

3.3 SCORE Y PRIORIDAD

El score es *probabilidad × impacto*, va de 1 a 9. La prioridad se deriva del score.

	Impacto		
	1 — Menor	2 — Degradado	3 — Crítico

↓ continúa en la página siguiente

	3 Probable	3 P3	6 P1	9 P0
	2 Posible	2 P3	4 P2	6 P1
	1 Improbable	1 P3	2 P3	3 P3

Cada celda mapea a una prioridad y a una acción esperada:

Score	Prioridad	Acción	Efecto en gate
9	P0 Critical Must Test	Bloquear release si falla.	FAIL automático.
6-8	P1 High Should Test	Mitigar antes del release.	CONCERNS.
4-5	P2 Medium Nice to Test	Monitorizar.	Sin efecto en gate.
1-3	P3 Low Test if Time Permits	Documentar y diferir.	Sin efecto en gate.

3.4 TARGETS DE COBERTURA POR PRIORIDAD

Prioridad	Unit	Integration	End-to-end	Tipo de tests
P0	> 90 %	> 80 %	todos los caminos críticos	happy + unhappy + edge + perf bajo carga
P1	> 80 %	> 60 %	main happy paths	happy primarios + errores clave + edge críticos
P2	> 60 %	> 40 %	smoke tests	happy path + error básico
P3	—	—	smoke opcional	acceptable manual; documentar limitaciones

3.5 AJUSTES QUE SUBEN O BAJAN LA PRIORIDAD

- **Sube:** > 50 % de usuarios afectados, > 10 K€ de pérdida potencial, riesgo de seguridad o cumplimiento, regresiones de bugs reportados, > 500 LOC, dependencias múltiples.
- **Baja:** feature flag con rollout gradual, observabilidad fuerte, rollback fácil, baja frecuencia de uso real.

ERROR FRECUENTE

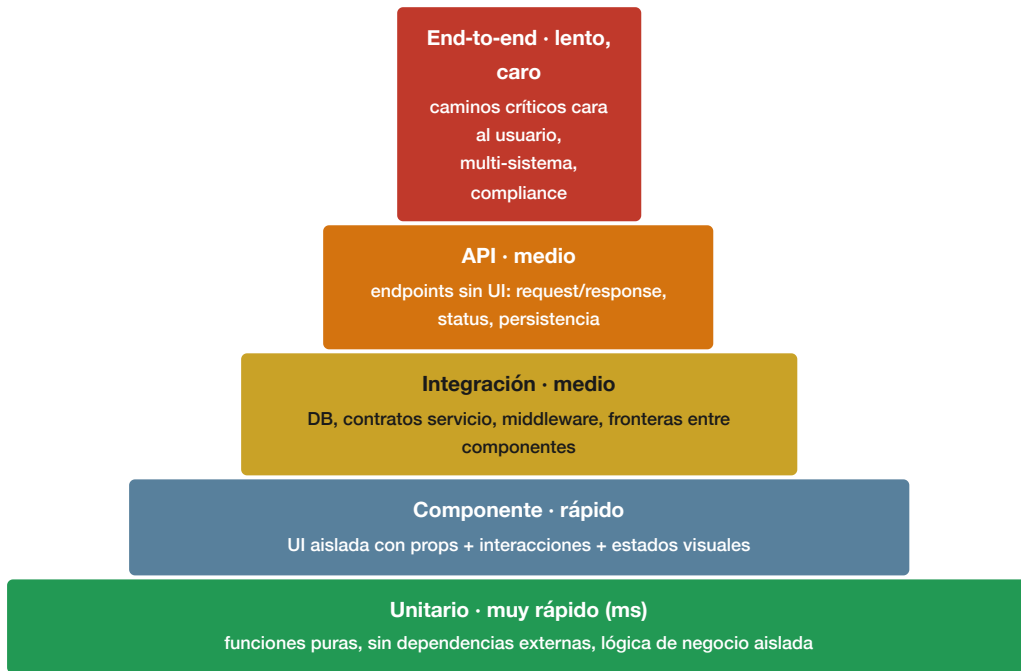
Confundir *cobertura* con *calidad*. Un proyecto puede tener 95 % de cobertura y dejar P0 críticos sin testear si la cobertura no se mide por riesgo. La pregunta correcta no es "¿qué porcentaje cubren los tests?" sino "¿qué P0 quedan sin cubrir?".

4 Niveles de test y cuándo elegir cada uno

Cada comportamiento debe probarse en el nivel más barato que lo cubra. La pirámide es el modelo: muchos tests rápidos abajo, pocos lentos y caros arriba.

MARCO

Pirámide de niveles de test



4.1 CUÁNDO ELEGIR CADA NIVEL

Favorece	Cuando...
Unit	la lógica es aislable, no hay efectos secundarios, se necesita feedback rápido, la complejidad ciclomática es alta.
Componente / Integración	se cruzan fronteras (persistencia, contratos, middleware) o la interacción entre piezas es lo que se quiere validar.
API	hay un endpoint público o interno y se quiere validar contrato y persistencia sin UI.
End-to-end	el camino atraviesa varios sistemas, es crítico para el usuario o impone compliance regulatorio o regresión visual.

4.2 SALVAGUARDA CONTRA COBERTURA DUPLICADA

Antes de añadir un test nuevo, comprobar tres preguntas. Si alguna se contesta "sí", reconsiderar el nivel:

1. ¿Está ya cubierto a un nivel inferior?
2. ¿Puede un unitario cubrir esto en lugar de un test de integración?
3. ¿Puede un test de integración cubrirlo en lugar de un end-to-end?

4.3 REGLAS API-FIRST PARA TESTS END-TO-END

- **Setup vía API, aserciones vía UI.** El setup por API es 10–50 veces más rápido que por UI.
- **Network-first interception** antes de la acción del usuario, para evitar carreras.
- **Saltar flujos innecesarios** con la API: verificación por email, signups multi-paso, semillas de datos.
- **Antipatrón:** validar lógica de negocio en end-to-end. Mover la validación a unit y dejar el end-to-end para el camino completo.

4.4 IDENTIFICADOR DE TEST

Formato canónico: {EPIC}. {STORY}–{LEVEL}–{SEQ}. Ejemplos: 1.3–UNIT–001, 1.3–INT–002, 1.3–E2E–001. Permite trazar de qué historia y nivel viene cada test.

5 Definition of Done de calidad de tests

Ocho criterios concretos que un test debe cumplir para considerarse "hecho". Tests que no los cumplen son deuda, no activo: degradan la suite y el equipo aprende a ignorar sus fallos.

MARCO Los 8 criterios — con ejemplo en cada uno

1. SIN ESPERAS DURAS

Usar `waitForResponse`, `waitForLoadState` o estado del elemento. Nunca `waitForTimeout`.

EVITAR

```
await page.waitForTimeout(3000);
```

HACER

```
await  
page.waitForResponse('*/api/dashboard');
```

2. SIN CONDICIONALES

El test ejecuta el mismo camino siempre. No `if/else` ni `try/catch` para control de flujo.

EVITAR

```
if (banner.isVisible()) banner.dismiss();
```

HACER

```
setup({ hasSeenWelcome: true });
```

3. MENOS DE 300 LÍNEAS

Si el test crece, dividir en escenarios focalizados y extraer setup a fixtures.

EVITAR

```
// 1 test de 400 líneas:  
// signup + create product + permissions +  
notifs
```

HACER

```
// 3 tests de ~60 líneas  
// con fixture adminPage compartido
```


4. MENOS DE 1 MINUTO Y MEDIO

Optimizar con setup por API, operaciones en paralelo y autenticación compartida.

EVITAR

```
// signup vía UI + product vía UI  
// total: 4 minutos
```

HACER

```
await Promise.all([  
  apiRequest.post('/users'),  
  apiRequest.post('/products'),  
]);  
// total: 45 segundos con storageState
```

5. AUTO-LIMPIEZA

Fixtures con cleanup automático tras `use()` o `afterEach()` explícito.

EVITAR

```
// email "newuser@example.com"  
// hardcoded sin cleanup
```

HACER

```
const email = faker.internet.email();  
createdUsers.push(email);  
// teardown borra el array entero
```

6. ASERCIONES EXPLÍCITAS

Las llamadas a `expect()` son visibles en el cuerpo del test, no escondidas en helpers.

EVITAR

```
validateUserCreation(response, email);  
// las aserciones quedan ocultas
```

HACER

```
expect(response.status()).toBe(201);  
expect(user.email).toBe(email);
```

7. DATOS ÚNICOS

Usar `faker` para datos dinámicos. Nunca hard-codear IDs ni emails.

EVITAR

```
const id = Math.random();  
const email = "test@example.com";
```

HACER

```
const id = faker.string.uuid();  
const email = faker.internet.email();
```

8. SEGURO EN PARALELO

Los tests no comparten estado y se ejecutan correctamente con `--workers=4`.

EVITAR

```
// test depende del orden:  
// si test_A no corre antes,  
// test_B falla
```

HACER

```
// cada test crea/limpia  
// sus propios datos  
// con IDs únicos
```

6 Decisión de gate

Antes de cada release, hito o cierre de épica, se firma una decisión. Cuatro estados posibles, cada uno con criterios concretos.

Decisión	Criterios	Qué implica para el release
PASS	Todos los tests verdes · cobertura por prioridad cumplida · sin hallazgos altos.	Release autorizado sin condiciones.
CONCERNS	Hallazgos de severidad media o ejes NFR ¹ que no cumplen umbral pero ninguno es bloqueante absoluto.	Release autorizado con riesgo documentado. Producto firma con conocimiento.
WAIVED	Existen huecos significativos pero el negocio acepta el riesgo con justificación escrita.	Release autorizado con ticket de deuda obligatorio (owner + deadline). Revisar la decisión en el siguiente cierre.
FAIL	P0 sin cobertura · tests rojos · eje NFR rompe compromiso contractual (SLA ² publicado, requisito regulatorio).	Release bloqueado. Resolver y volver a evaluar.

6.1 SEVERIDAD DE HALLAZGOS

El triage durante `/bmad-code-review` y `/bmad-testarch-test-review` clasifica cada hallazgo en una de tres severidades:

- **Alta:** bloquea o degrada un P0; produce FAIL si no se resuelve.
- **Media:** afecta a P1 o introduce deuda visible; produce CONCERNS.
- **Baja:** mejora opcional, no afecta a la decisión.

6.2 DEFAULT ANTE AMBIGÜEDAD

CUANDO NO ESTÁ CLARO

Si los datos no permiten decidir entre PASS y CONCERNS, la decisión por defecto es **CONCERNS**. Forzar al equipo a clarificar antes de firmar baja drásticamente el riesgo de releases con problemas no detectados.

7 Traceability viva

La trazabilidad entre requisitos y tests no es un documento que se rellena al final. Es un mapa que crece historia a historia y permite responder dos preguntas: ¿qué requisito cubre este test? y ¿qué tests cubren este requisito?

2. NFR: Non-Functional Requirement. Requisito sobre cómo de bien debe comportarse el sistema (rendimiento, seguridad, fiabilidad, escalabilidad), en contraposición a los requisitos funcionales que describen qué debe hacer.

2. SLA: Service Level Agreement. Acuerdo formal entre el proveedor de un servicio y sus usuarios sobre el nivel de calidad esperado (disponibilidad, latencia, etc.). El SLO interno debe ser más estricto que el SLA público.

7.1 LA REGLA

Cada vez que `/bmad-testarch-atdd` genera tests, los ata al requisito funcional o al criterio de aceptación correspondiente. Cada test nuevo entra a la matriz inmediatamente.

7.2 POR QUÉ IMPORTA

Si la matriz se construye al cierre de la épica (en lugar de mantenerse historia a historia), el ejercicio se vuelve arqueología: alguien intenta reconstruir qué test cubre qué requisito a partir del nombre del archivo, y se generan huecos invisibles. La matriz "viva" evita esto:

- El gate al cierre de épica solo confirma; no construye.
- Los huecos de cobertura se ven en cuanto aparecen, no semanas después.
- Cuando un requisito cambia, se sabe qué tests revisar.

7.3 CÓMO SE MANTIENE

La skill `/bmad-testarch-atdd` escribe en el archivo de la historia una sub-sección con la lista de tests generados y su mapping a criterios de aceptación. Al cierre de la épica, `/bmad-testarch-trace` agrega esos mappings en la matriz definitiva, detecta huecos y emite la decisión de gate.

PARTE B

Las skills TEA en uso

Mapa rápido y nueve fichas detalladas. Cada ficha cita los marcos de la Parte A en lugar de redefinirlos. Para el conteo exacto de sub-pasos y modos CEV, ver el Libro 4 Sec. 2.

8 Mapa de las skills TEA

8.1 Las nueve skills de un vistazo

Skill	Cuándo	Obligatoriedad	Aplica los marcos
/bmad-teach-me-testing	Pre-adopción	OPC	Vocabulario base de Sec. 3, Sec. 4 y Sec. 5.
/bmad-testarch-test-design	Fase 3	REQ TEA	Aplica Sec. 3 (riesgo) + Sec. 4 (niveles) + Sec. 7 (traceability).
/bmad-testarch-framework	Fase 3	REQ TEA	Infraestructura; respeta Sec. 5 (DoD) en sus plantillas.
/bmad-testarch-ci	Fase 3	REQ TEA	Aplica Sec. 5 (DoD) y burn-in para detectar inestabilidad.
/bmad-testarch-atdd	Fase 4 ciclo	REQ TEA	Aplica Sec. 3 + Sec. 4 + Sec. 5 + Sec. 7.
/bmad-testarch-automate	Fase 4 ciclo	REC	Aplica Sec. 4 + Sec. 5.
/bmad-testarch-test-review	Fase 4 cierre	OPC	Aplica Sec. 5 + Sec. 6.
/bmad-testarch-trace	Fase 4 cierre	OPC	Aplica Sec. 6 + Sec. 7.
/bmad-testarch-nfr	Fase 2 + 4	REC	Tres modos — setup, check, gate — aplica Sec. 6.

8.2 Regla bloqueante de orden

DEPENDENCIAS ESTRICTAS, NO NEGOCIAN

El bloque de arranque tiene un orden fijo: `test-design` → `framework` → `ci`. Y dentro del ciclo: `framework` instalado antes de `atdd`; `atdd` antes de `dev-story`. Saltarse cualquier dependencia genera un fallo silencioso: la skill posterior se ejecuta pero produce salida no útil.

Modos uniformes en todas las skills `testarch-*`: **Create** (genera el artefacto), **Edit** (modifica el existente), **Validate** (audita sin tocarlo). Detalle en el Libro 4 Sec. 0.2.

9

/bmad-teach-me-testing

01

/bmad-teach-me-testing

● Test Architect

OPC

PRE-ADOPCIÓN

RESUELVE: Forma al equipo en disciplina de testing antes de adoptar TEA, con un programa autodirigido y reanudable.

Programa de siete sesiones académicas (2–4 h cada una) con persistencia de progreso. Pensado para equipos sin disciplina previa de testing. Proporciona el vocabulario común que usan Sec. 3, Sec. 4 y Sec. 5.

Ficha completa en el **Libro 4 (Catálogo de Skills) Sec. 2.0**. La encajamos en el calendario LIDR como sesión preliminar S0 del Libro 3.

10

/bmad-testarch-test-design

02

/bmad-testarch-test-design

● Test Architect

REQ TEA

FASE 3

RESUELVE: Plan de testing basado en riesgo con clasificación P0–P3, criterios de entrada y salida y orden de ejecución.

MARCOS QUE APLICA

- Sec. 3 Probability×Impact → prioridades P0–P3.
- Sec. 4 Niveles → nivel correcto por requisito.
- Sec. 7 Traceability → bootstrap de la matriz.

SALIDA

Plan de testing y matriz de riesgos. Plantillas: system, epic, qa, handoff y architecture.

AUTO-DETECTA EL MODO

- *system-level*: Fase 3, valida testabilidad arquitectónica.
- *epic-level*: Fase 4, plan por épica.

GATE DE TESTABILIDAD POST-ARQUITECTURA

Antes de empezar, aplicar el ADR Quality Readiness Checklist (Sec. 24.4): 29 criterios en 8 categorías.

11

/bmad-testarch-framework

03

/bmad-testarch-framework

● Test Architect

REQ TEA

FASE 3

RESUELVE: Andamiaje del framework de tests listo para producción con fixtures, helpers y configuración alineados al Definition of Done.

SOPORTA

Playwright o Cypress según preferencia. Configurable por tamaño de proyecto y uso de TypeScript.

MARCOS QUE APLICA

Sec. 5 (DoD) en sus plantillas: tests por defecto cumplen ya los ocho criterios.

LO QUE DEJA LISTO

`playwright.config.ts` o `cypress.config.ts`; fixtures, helpers, page objects, scripts y README de setup.

DEPENDENCIA BLOQUEANTE

Debe quedar instalado antes del primer ATDD. Sin framework, ATDD genera pseudocódigo no ejecutable.

12

/bmad-testarch-ci

04

/bmad-testarch-ci

● Test Architect

REQ TEA

FASE 3

RESUELVE: Pipeline de integración continua multi-plataforma con ejecución de tests, ciclos de burn-in para detectar tests inestables, sharding paralelo, recolección de artefactos y notificaciones.

PLATAFORMAS SOPORTADAS

GitHub Actions, GitLab CI, Azure Pipelines, Jenkins, Harness.

MARCOS QUE APLICA

Sec. 5 (DoD) — los tests inestables fallan burn-in y no llegan a main.

BURN-IN

Corre los tests cambiados N veces (típico 10) para detectar inestabilidad antes del merge. 10/10 estable; cualquier fallo bloquea el merge.

DEPENDENCIA BLOQUEANTE

Pipeline debe estar configurada antes del primer merge en Fase 4.

13

/bmad-testarch-atdd

05

/bmad-testarch-atdd

● Test Architect

REQ TEA

FASE 4 · CICLO

RESUELVE: Genera tests de aceptación que fallan (fase roja) antes de la implementación, en el nivel correcto, atados al requisito que cubren.

MARCOS QUE APLICA

- Sec. 3 prioriza el riesgo del requisito.
- Sec. 4 elige nivel correcto (E2E, API, componente).
- Sec. 5 los tests cumplen DoD desde el nacimiento.
- Sec. 7 traceability — cada test ata al criterio de aceptación.

SALIDA

Tests fallidos en niveles apropiados + fixtures y helpers + checklist de implementación + mapping criterio↔test.

SUB-AGENTES PARALELOS

Genera tests por nivel en paralelo (API, E2E) usando sub-agentes especializados.

CUÁNDO INVOCARLA

Tras `/bmad-create-story`, antes de `/bmad-dev-story`. Punto de entrada al ciclo TDD³.

14

`/bmad-testarch-automate`

06

`/bmad-testarch-automate`

● Test Architect

REC

FASE 4 · CICLO

RESUELVE: Expansión de la cobertura tras la implementación: genera tests adicionales priorizados a nivel unit, componente, API y end-to-end.

MARCOS QUE APLICA

- Sec. 4 niveles — cubre lo que ATDD no cubrió en el nivel adecuado.
- Sec. 5 DoD — cada test nuevo cumple los ocho criterios.

SALIDA

Tests por nivel + reporte de cobertura (actual contra objetivo) + cola de implementación con prioridades.

DOS MODOS

- *BMad-Integrated*: usa historia, PRD y test design.
- *Standalone*: analiza el código existente sin artefactos BMad — útil en proyectos brownfield.

ANTIPATRÓN

Sustituir esta skill por code review humano. Ambas se complementan: code review captura defectos, automate añade red.

15

`/bmad-testarch-test-review`

07

`/bmad-testarch-test-review`

● Test Architect

OPC

FASE 4 · CIERRE

RESUELVE: Auditoría de calidad del conjunto de tests con puntuación 0–100 y hallazgos accionables por dimensión.

3. TDD: Test-Driven Development. Disciplina de desarrollo en la que los tests se escriben antes que el código de producción. ATDD es la variante en la que los tests se derivan de los criterios de aceptación de la historia.

DIMENSIONES EVALUADAS

- Determinismo (¿pasa siempre 10/10?).
- Aislamiento (¿depende de orden o estado compartido?).
- Mantenibilidad (¿es legible y refactorizable?).
- Rendimiento (¿corre en menos del umbral?).

SALIDA

Reporte con puntuación 0–100, hallazgos por dimensión y severidad, lista priorizada de tests a refactorizar o eliminar.

MARCOS QUE APLICA

Sec. 5 (cada hallazgo cita qué criterio del DoD se incumple) y Sec. 6 (alimenta la decisión de gate).

CUÁNDO INVOCARLA

Cierre de épica (paso 1), pre-release, auditoría trimestral, sospecha de inestabilidad.

16

/bmad-testarch-trace

08

/bmad-testarch-trace

● Test Architect

OPC

FASE 4 · CIERRE

RESUELVE: Matriz de trazabilidad requisitos↔tests, análisis de huecos y decisión opcional de gate de calidad basada en evidencia. Salida go/no-go.

MARCOS QUE APLICA

- Sec. 7 traceability — agrega lo que ATDD viene escribiendo historia a historia.
- Sec. 6 gate decision — emite PASS / CONCERNS / WAIVED / FAIL.

SALIDA

Matriz de trazabilidad + reporte de cobertura + decisión de gate firmada.

CUÁNDO INVOCARLA

Cierre de épica (paso 2, después del test review), pre-release, auditoría regulatoria.

RECOMENDACIÓN DE ORDEN

Ejecutar siempre tras /bmad-testarch-test-review .
La puntuación de calidad alimenta la decisión de gate.

17

/bmad-testarch-nfr

09

/bmad-testarch-nfr

● Test Architect

REC

FASE 2 & 4

RESUELVE: Evalúa los requisitos no funcionales con validación basada en evidencia y desenlace determinista — sin sensaciones, sin "el sistema debe ser rápido".

VISIÓN LIDR

En el método, NFR es una sola skill con tres modos. Nosotros la pensamos como tres momentos del proyecto: *setup* en Fase 2 para fijar umbrales, *check* a mitad del ciclo para detectar deriva, *gate* antes del release como decisión final. Cada momento responde una pregunta distinta.

17.1 LOS CUATRO EJES Y UMBRALES TÍPICOS

Eje	Qué evalúa	Umbral es típicos
Seguridad	Autenticación, autorización (RBAC), token expiry, secret handling, OWASP Top 10.	JWT expira en 15 min · passwords nunca en logs ni DOM ni consola · SQL injection bloqueada (no crash) · XSS sanitizado · RBAC retorna 403, no 200.
Rendimiento	SLO ⁴ y SLA validados con load, stress, spike y soak tests. Playwright solo para perceived performance (Core Web Vitals).	P95 ⁵ de petición HTTP < 500 ms · P99 de API < 1000 ms · tasa de error < 1 % · soporta ramp 1 m hasta 50 RPS ⁶ + sustain 3 m + spike 1 m hasta 100 + ramp down 1 m.
Fiabilidad	Manejo de errores, retries, health checks, circuit breakers, offline, rate limiting.	Error 500 → mensaje friendly + botón retry (no blanco ni stack trace) · 3 reintentos en 503 transitorio · circuit breaker abre tras 5 fallos consecutivos · <code>/api/health</code> retorna 200 con DB, cache y queue UP · 429 respeta <code>Retry-After</code> .
Mantenibilidad	Calidad de código medida en herramientas de CI — no se valida con Playwright.	Cobertura ≥ 80 % · duplicación < 5 % (jscpd) · cero vulnerabilidades critical o high (npm audit) · logging estructurado con trace IDs (<code>X-Trace-Id</code>) · error tracking activo · header <code>Server-Timing</code> con <code>db</code> y <code>total</code> .

17.2 LOS TRES MODOS EN USO

Modo	Cuándo	Salida
Setup	Fase 2 — tras PRD.	Umbral es numéricos por eje, persistidos en el PRD. Sin esto, gate no tiene contra qué validar.
Check	Mid-ciclo — tras cambios de infraestructura, dependencias o arquitectura.	Reporte intermedio: ¿vamos camino de cumplir? Si no, qué desviación corregir.
Gate	Pre-release.	Firma final por eje y agregada: PASS, CONCERNS, WAIVED o FAIL. FAIL bloquea release.

6. **SLO:** *Service Level Objective*. Objetivo cuantificable de calidad de servicio que el equipo se compromete a cumplir — por ejemplo, "el 99,9 % de las peticiones responde en menos de 200 ms".

6. **P95:** percentil 95 de latencia. Significa que el 95 % de las peticiones responden en un tiempo igual o inferior al valor declarado.

6. **RPS:** *requests per second*. Métrica de carga que indica cuántas peticiones por segundo soporta un sistema.

PARTE C

Escenarios cross-cutting

Cuatro patrones de uso reales que aparecen en proyectos en marcha. Cada uno aplica varios marcos y skills de las partes anteriores. Pensados como runbooks operativos: si te pasa lo descrito, abres la sección y sigues los pasos.

18 Hotfix — producción urgente

ESCENARIO 1

Incidente en producción que requiere fix inmediato

REGLA MAESTRA

Un hotfix no exime de la calidad: la reordena en el tiempo. La meta es minimizar el tiempo durante el cual el bug afecta a usuarios sin acumular deuda oculta.

LO QUE NO SE PUEDE SALTAR

- Reproducción local con test rojo antes de tocar código de producción.
- Fix mínimo. Solo lo necesario para que el test pase. Sin refactors "de paso".
- Code review (un Tech Lead, sincrónico o verbal aceptable).
- Smoke test en pre-producción.
- Audit log: quién, cuándo, por qué.

LO QUE SE PUEDE DIFERIR (CON TICKET OBLIGATORIO)

- Tests adicionales y casos de borde → pull request separado en 48-72 h vía `/bmad-testarch-automate`.
- ADR⁷ y documentación → siguiente sprint con ticket.
- Retrospectiva del incidente → asíncrona en cinco días.
- Actualización de Test Design si el bug reveló área no cubierta → al cierre de la siguiente épica.

CRONOGRAMA RECOMENDADO

T+0	Incidente reportado.
T+5 m	Confirmación en producción (ping a oncall o Tech Lead).
T+15 m	Test rojo que reproduce el bug, en local.
T+30 m	Fix mínimo, test pasa local.
T+45 m	PR de emergencia + code review (sincrónico si crítico).

7. ADR: *Architecture Decision Record*. Documento corto que captura una decisión técnica (qué se eligió, qué se descartó y por qué), con sus trade-offs.

T+60 m Merge + deploy a pre-producción + smoke.

T+75 m Deploy a producción + monitoreo activo.

T+90 m Incidente cerrado.

T+24 h PR de seguimiento con tests adicionales.

T+48 h Documentación o ADR si aplica.

T+5 d Retrospectiva del incidente.

TRAMPAS COMUNES A EVITAR

"Después escribimos el test" — nunca llega. "Aprovechamos para refactorizar" — el PR crece y deja de ser hotfix. "Saltamos code review por urgencia" — introduce regresiones en frío. "Es trivial, no necesita smoke" — cuando algo es trivial es justo cuando se rompe en producción.

19 Cuarentena de tests inestables

ESCENARIO 2 Tests que pasan unas veces sí y otras no

PASOS OPERATIVOS

1. **Confirmar la inestabilidad con burn-in.** Correr el test 10 veces. 10/10 verdes = posible incidencia única. 1–4 fallos = inestable. 5 o más = roto.
2. **Clasificar por nivel.** Determinístico (0/10), sospechoso (1/10), inestable (2–4/10), roto ($\geq 5/10$).
3. **Cuarentena técnica.** Marcar con `test.skip`, `xit`, `pytest.mark.skip` o `@Disabled`. Nunca borrar sin ticket.
4. **Ticket obligatorio.** Título "FLAKY: <nombre del test>", body con los últimos tres fallos, hipótesis, owner y deadline (cierre del sprint siguiente).
5. **Recalcular cobertura efectiva** excluyendo el cuarentenado. La métrica no debe quedar inflada.
6. **Resolver o eliminar dentro del plazo.** Al cierre del sprint el ticket se cierra de una manera u otra; no se prorroga.

HIPÓTESIS MÁS COMUNES Y SOLUCIÓN

Causa	Solución
Race condition en operación asíncrona	Reemplazar <code>setTimeout</code> por <code>waitFor</code> o <code>expect.poll</code> : esperar la condición, no el tiempo.
Dependencia de orden de tests	Aislar estado: cada test crea y limpia sus datos, sin depender de tests previos.
Datos compartidos entre tests	Data factories con IDs únicos por test (<code>faker</code>).

↓ continúa en la página siguiente

Inestabilidad de red	Interceptar o mockear las llamadas en el límite correcto. Network-first.
Selectores frágiles	<code>data-testid</code> o roles ARIA, no clases CSS ni xpath posicional.
Animaciones	Deshabilitar animaciones en el entorno de tests.
Fechas y zonas horarias	Fake timers con fecha fija para toda la suite.
Escrituras concurrentes en BD	Aislar BD por worker (uno por proceso paralelo).

20 Auditoría de suite legacy (brownfield)

ESCENARIO 3 Equipo nuevo recibe un proyecto con tests existentes

Cuando se adopta TEA en un proyecto que ya tiene suite, lo primero no es invocar test-design ni framework: es auditar lo que hay. Cuatro etapas, en orden.

ETAPA 1 — INVENTARIO (1-2 DÍAS)

Produce: conteo y clasificación por nivel (detecta pirámide invertida); mapa de cobertura por módulo o dominio (no número global, áreas críticas con <50 % son P0); tiempo total de la suite (>30 min sugiere pirámide invertida); conteo de tests *skipped* o *disabled* (cualquier `.skip` commiteado es deuda); línea base de inestabilidad (correr la suite cinco veces); línea base de NFR vía `/bmad-testarch-nfr` en modo check.

ETAPA 2 — EVALUACIÓN DE CALIDAD (2-3 DÍAS)

Muestra de 20-30 tests por nivel evaluada contra el DoD (Sec. 5). Clasificar tests problemáticos:

- **Tautológicos:** copia-pegar de la implementación en las aserciones → reescribir o eliminar.
- **Acoplados a implementación:** rompen al refactorizar sin cambiar comportamiento → refactorizar a comportamiento observable.
- **Sin aserciones:** solo "no throw" → añadir aserciones o eliminar.
- **Gigantes** (>100 líneas) → dividir en tests focalizados.
- **Literales inline:** mock data hardcodeado y duplicado → extraer a data factories.

ETAPA 3 — GAPS CONTRA RISK MATRIX (1-2 DÍAS)

Aplicar el marco Sec. 3 a cada épica del producto. Roadmap por área:

- **P0** con cobertura <50 % → sprint dedicado, `/bmad-qa-generate-e2e-tests` retroactivo, bloquear features hasta cerrar.
- **P0** con 50-80 % → cobertura incremental, target 90 % en 2-3 sprints.
- **P1** con <50 % → próximo sprint, no bloquea features.
- **P2** y **P3** sin cobertura → aceptable, smoke opcional.

ETAPA 4 — DECISIÓN DE GATE Y ROADMAP (MEDIO DÍA)

Aplicar Sec. 6 (Gate Decision). Salidas posibles:

- **PASS** (raro): la suite cumple el DoD y cubre P0.
- **CONCERNS** (típico): sprints incrementales para cerrar huecos.
- **WAIVED**: huecos significativos pero el negocio acepta el riesgo. Roadmap 6–12 semanas, revisión cada dos sprints.
- **FAIL**: suite roto o pirámide invertida grave. Freeze de features + sprint de limpieza obligatorio.

Output esperado: documento de 2–4 páginas con inventario + evaluación + gaps + decisión de gate + roadmap a 3–6 sprints.

ERROR FRECUENTE AL ADOPTAR TEA EN BROWNFIELD

Saltarse la auditoría y aplicar TEA "a partir de ahora". El flujo asume una línea base mínima de calidad; si la suite anterior tiene tests inestables y pirámide invertida, ATDD se ejecuta sobre suelo movedizo y produce más ruido que señal. Auditar primero, adoptar después.

21 Diagnóstico rápido — pitfalls comunes

ESCENARIO 4 Tres situaciones diagnósticas

21.1 "EL EQUIPO ADOPTÓ TEA PERO LOS TESTS NUNCA CORREN EN CI"

Causa más probable: CI Setup quedó a medias — scaffolding sin wiring. El job de la pipeline se creó pero solo ejecuta install, sin step de tests.

Acción: re-ejecutar `/bmad-testarch-ci`. Confirmar que la pipeline incluye install → lint → ejecución de tests → reporte. Verificar que el step de tests *falla* el job (sin `continue-on-error: true`).

21.2 "ATDD GENERA TESTS QUE SIEMPRE PASAN DESDE EL INICIO"

Causa más probable: el test no cubre comportamiento real. Aserciones vacuas (`expect(true).toBe(true)`), mocks que devuelven exactamente lo que el test verifica, o ausencia de fase roja del ciclo TDD.

Acción: aplicar el DoD (Sec. 5). Si el test pasa antes de implementar el código, no verifica comportamiento — reescribir derivándolo del criterio de aceptación de la historia.

21.3 "EL GATE NFR PRE-RELEASE DEVUELVE CONCERNS"

Causa más probable: uno o varios ejes (rendimiento, seguridad, fiabilidad, mantenibilidad) no cumple umbral, pero ninguno es bloqueante absoluto. CONCERNS = riesgo documentado, no compromiso roto.

Tres acciones posibles:

- **Fix antes del release** si el eje es crítico para el caso de uso real (por ejemplo, P95 de latencia en checkout).

- **Aceptar como WAIVED** con justificación escrita y ticket de deuda con owner y deadline, si es aceptable temporalmente.
 - **Convertir en FAIL** si rompe un compromiso contractual: SLA publicado o requisito regulatorio.
-

Recursos

Persona Murat, listado del knowledge base, plantillas listas para imprimir y verificación de fuentes.

22 Murat — el agente Test Architect

Cuando una situación no encaja con ninguna skill concreta, o cuando el equipo necesita criterio en zona gris, se invoca a Murat: el agente con persona del módulo TEA. No produce un artefacto único; conversa.

22.1 Voz y enfoque

Murat es *data-driven*: habla en cálculos de riesgo y evaluaciones de impacto. Su lema es "opiniones fuertes, sostenidas con pinzas". Mezcla rigor cuantitativo con instinto experto. Persiste cuando se invoca otra skill TEA: vuelves a Murat al cerrar la skill hija.

22.2 Menú de capabilities

Cuando se activa, Murat presenta un menú de nueve capacidades. Cada una corresponde a una skill TEA concreta o a una consulta directa del knowledge base.

Código	Capability	Cuándo pedirla
TMT	Teach Me Testing	Onboarding de devs, vocabulario compartido pre-TEA.
TF	Test Framework setup	"Qué tooling instalo para mi stack?".
AT	ATDD	"Cómo genero tests rojos para esta historia?".
TA	Test Automation	"Cómo expando cobertura tras Dev Story?".
TD	Test Design	"Aplica matriz de riesgo a esta épica".
TR	Test Review	"Audita la calidad del suite actual".
NR	NFR Assessment	"Evalúa los cuatro ejes ante el release".
CI	CI Setup	"Configura pipeline con gates por prioridad".
RV	Trace o Coverage Review	"Genera matriz de requisitos contra tests".

22.3 Cuándo Murat aporta más que una skill aislada

Murat carga el conocimiento contextual del knowledge base completo (fixture architecture, contract testing, ADR Quality Readiness Checklist, networking, etc.). Las skills atómicas no lo cargan por defecto. Cuando la consulta es "qué patrón aplica aquí", Murat es la entrada correcta; cuando la consulta es "haz X", la skill atómica es más eficiente.

23 Knowledge base — lo que las skills consultan

Cada skill TEA carga un conjunto de archivos de conocimiento al ejecutarse. Estos archivos no son código: son guías técnicas concretas (cómo gestionar fixtures, cómo interceptar llamadas de red, cómo aplicar contract testing) que el agente usa para tomar decisiones informadas. La carpeta canónica está en `.claude/skills/bmad-tea/resources/knowledge/`.

23.1 Catálogo (50 archivos)

Archivo	Para qué sirve
<code>adr-quality-readiness-checklist.md</code>	29 criterios en 8 categorías para evaluar testabilidad y NFR de un ADR.
<code>api-request.md</code>	Patrones de petición a API en tests (Playwright, Cypress).
<code>api-testing-patterns.md</code>	Patrones de testing de APIs: request/response, validación, status codes.
<code>auth-session.md</code>	Reuso de sesiones de auth vía <code>storageState</code> o <code>setCookie</code> .
<code>burn-in.md</code>	Selección inteligente de tests con filtros y volumen controlado.
<code>ci-burn-in.md</code>	Loop tradicional de burn-in en pipeline (10 iteraciones).
<code>component-tdd.md</code>	Ciclo TDD a nivel de componente UI.
<code>contract-testing.md</code>	Conceptos de contract testing (Pact CDC).
<code>data-factories.md</code>	Factories de datos para tests aislados con identificadores únicos.
<code>email-auth.md</code>	Testing de flujos de autenticación por email.
<code>error-handling.md</code>	Patrones de validación de error handling en tests.
<code>feature-flags.md</code>	Governance de feature flags en tests.
<code>file-utils.md</code>	Utilidades de manejo de archivos en tests.
<code>fixture-architecture.md</code>	Arquitectura de fixtures con auto-cleanup.
<code>fixtures-composition.md</code>	Composición de fixtures con la API <code>extends</code> de Playwright.
<code>intercept-network-call.md</code>	Interceptación de llamadas de red en tests.
<code>log.md</code>	Logging y observabilidad en tests.
<code>network-error-monitor.md</code>	Monitor de errores de red durante la ejecución.

↓ continúa en la página siguiente

<code>network-first.md</code>	Patrón <i>network-first</i> : interceptar antes de navegar.
<code>network-recorder.md</code>	Grabación y reproducción de tráfico HAR.
<code>nfr-criteria.md</code>	Criterios NFR detallados (seguridad, rendimiento, fiabilidad, mantenibilidad).
<code>overview.md</code>	Visión general e instalación del knowledge base.
<code>pact-broker-webhooks.md</code>	Webhooks del Pact Broker.
<code>pact-consumer-di.md</code>	Inyección de dependencias en consumer de Pact.
<code>pact-consumer-framework-setup.md</code>	Setup de Pact en consumer frameworks.
<code>pact-mcp.md</code>	Integración Pact con servidores MCP.
<code>pactjs-utils-consumer-helpers.md</code>	Helpers de pact-js para consumer.
<code>pactjs-utils-overview.md</code>	Visión general de utilities de pact-js.
<code>pactjs-utils-provider-verifier.md</code>	Verifier del provider en pact-js.
<code>pactjs-utils-request-filter.md</code>	Filtros de petición para pact.
<code>pactjs-utils-zod-to-pact.md</code>	Conversión de schema zod a matchers pact.
<code>playwright-cli.md</code>	CLI de Playwright (comandos clave).
<code>playwright-config.md</code>	Configuración de Playwright (workers, timeouts, projects).
<code>probability-impact.md</code>	Matriz de riesgo 3x3, score 1–9, mapeo a P0–P3 (Sec. 3).
<code>recurse.md</code>	Patrón de retry y recursión en tests.
<code>risk-governance.md</code>	Risk scoring matrix y motor de decisión de gate.
<code>selective-testing.md</code>	Selección de tests por etiqueta o archivos cambiados.
<code>selector-resilience.md</code>	Selectores resilientes (data-testid, roles ARIA).
<code>test-healing-patterns.md</code>	Auto-healing de tests rotos.
<code>test-levels-framework.md</code>	Niveles unit / componente / integración / API / E2E + reglas de elección (Sec. 4).
<code>test-priorities-matrix.md</code>	P0–P3 con criterios y targets de cobertura (Sec. 3).
<code>test-quality.md</code>	Los 8 criterios del Definition of Done (Sec. 5).
<code>timing-debugging.md</code>	Debugging de problemas de timing.
<code>visual-debugging.md</code>	Debugging visual: traces, screenshots, vídeo.

↓ continúa en la página siguiente

<code>webhook-module-setup.md</code>	Setup del módulo de webhooks.
<code>webhook-providers.md</code>	Providers de webhooks soportados.
<code>webhook-risk-guidance.md</code>	Guía de riesgo en testing de webhooks.
<code>webhook-template-matchers.md</code>	Template matchers para webhooks.
<code>webhook-testing-fundamentals.md</code>	Fundamentos de testing de webhooks.
<code>webhook-timeout-error.md</code>	Manejo de timeouts y errores en webhooks.
<code>webhook-waiting-querying.md</code>	Estrategias de espera y consulta de webhooks.

NOTAS SOBRE EL CATÁLOGO

Las nueve skills `bmad-testarch-*` reusan el mismo set de 49 archivos. La carpeta de `bmad-tea` incluye uno adicional (`pactjs-utils-zod-to-pact.md`), totalizando 50. El listado anterior es el de `bmad-tea`, el más completo.

24 Plantillas operativas

Cuatro plantillas listas para usar. Cada una refleja un marco o un escenario del libro y se puede copiar a un documento del proyecto.

24.1 Risk Matrix — plantilla por épica

```
# Risk Matrix - Épica <N>

| ID | Riesgo | Probabilidad (1-3) | Impacto (1-3) | Score | Prioridad | Mitigación |
|----|-----|-----|-----|-----|-----|-----|
| R-01 | <descripción> | 2 | 3 | 6 | P1 | <test específico> |
| R-02 | <descripción> | 1 | 3 | 3 | P3 | documentar y monitor |
| R-03 | <descripción> | 3 | 3 | 9 | P0 | bloquear release si falla |

## Criterios de entrada (sprint)
- <criterio 1>
- <criterio 2>

## Criterios de salida (release)
- Todos los P0 cubiertos a niveles unit, integración y E2E.
- P1 cubiertos en main happy path.
- Decisión de gate firmada (PASS / CONCERNS / WAIVED / FAIL).
```

24.2 NFR Criteria — plantilla de setup en S1

Acuerdo explícito de los cuatro NFRs core (seguridad, rendimiento, fiabilidad, mantenibilidad) en S1 de cada épica. Sin esta plantilla, los NFRs son ambiguos hasta el final y suelen fallar en el gate.

```
# NFR Criteria – Épica <N>

## Seguridad
target: <ej. OWASP ASVS L2 cumplido en endpoints expuestos>
medición: <cómo se verifica: scan automatizado, pen-test, code review>
gate_threshold: 0 vulnerabilidades High/Critical en main

## Rendimiento
target: <ej. P95 < 200 ms en endpoints <X, Y, Z> bajo 100 RPS>
medición: <k6, Lighthouse, APM real-user metrics>
gate_threshold: P95 < SLA pactado, sin regresiones > 10% vs baseline

## Fiabilidad
target: <ej. tasa de error < 0.1% en main flows; recuperación < 30 s>
medición: <chaos tests, error budget tracking, observabilidad>
gate_threshold: error budget consumido < 50% en la épica

## Mantenibilidad
target: <ej. complejidad ciclomática < 10; cobertura branches > 80% en código nuevo>
medición: <SonarQube, ESLint, code review checklist>
gate_threshold: zero código sin tests en módulos P0/P1
```

24.3 Trace Matrix — plantilla AC → test (Given/When/Then)

Trazabilidad viva: cada criterio de aceptación de una historia mapea a uno o más tests, expresados en sintaxis Given/When/Then para que el lenguaje sea compartido entre PO, dev y TEA.

```
# Trace Matrix – Historia <HU-NN>

## AC-1: <descripción del criterio de aceptación>
test_id: T-NN-01
nivel: <unit | integration | e2e>
prioridad: <P0 | P1 | P2 | P3>
Given: <contexto/estado inicial>
When: <acción del actor>
Then: <resultado observable>
archivo: <path/to/test.spec.ts>
estado: <PASS | FAIL | NOT_RUN>

## AC-2: <descripción>
test_id: T-NN-02
nivel: <...>
prioridad: <...>
Given: <...>
When: <...>
Then: <...>
archivo: <...>
estado: <...>

## Cobertura
total_ACs: <N>
ACs_cubiertos: <M>
gaps: <lista de ACs sin test, con justificación o ticket>
```

24.4 ADR Quality Readiness Checklist — 29 criterios en 8 categorías

Aplicar a cada ADR antes de iniciar Test Design. Cada criterio puntúa 1 si se cumple, 0 si no. Score = X/29. Decisión: PASS si ≥ 25 , CONCERNS si 20–24, FAIL si < 20 .

```
# ADR Quality Readiness Checklist – <ADR-NN>

## 1. Testabilidad y automatización (4)
[ ] 1.1 Isolation – el servicio se puede testear con todas las deps mockeadas
[ ] 1.2 Headless interaction – 100% de la lógica accesible vía API
[ ] 1.3 State control – APIs/scripts para inyectar estados específicos
[ ] 1.4 Sample requests – cURL/JSON válidos e inválidos en el doc

## 2. Estrategia de datos de test (3)
[ ] 2.1 Segregation – multi-tenancy o headers para no contaminar prod
[ ] 2.2 Generation – datos sintéticos o scrubbing PII/RGPD
[ ] 2.3 Teardown – reset y cleanup post-test destructivo

## 3. Escalabilidad y disponibilidad (4)
[ ] 3.1 Statelessness – el servicio es stateless o replica session
[ ] 3.2 Bottlenecks – weak link bajo carga identificado
[ ] 3.3 SLA definitions – target de availability + redundancia
[ ] 3.4 Circuit breakers – fail fast si dep falla

## 4. Disaster recovery (3)
[ ] 4.1 RTO/RPO definidos
[ ] 4.2 Failover automático o manual, practicado
[ ] 4.3 Backups inmutables, restoration testeada

## 5. Seguridad (4)
[ ] 5.1 AuthN/AuthZ – OAuth2/OIDC + Least Privilege granular
[ ] 5.2 Encryption – at rest + in transit (TLS)
[ ] 5.3 Secrets – en Vault, no en código/config
[ ] 5.4 Input validation – sanitización contra SQLi/XSS

## 6. Monitorabilidad y manageability (4)
[ ] 6.1 Tracing – W3C Trace Context propagado
[ ] 6.2 Logs – log levels toggle dinámico sin redeploy
[ ] 6.3 Metrics – RED metrics expuestas (Prometheus, Datadog)
[ ] 6.4 Config – externalizada, cambios sin rebuild

## 7. QoS y QoE (4)
[ ] 7.1 Latency – targets P95 y P99
[ ] 7.2 Throttling – rate limiting contra DDoS
[ ] 7.3 Perceived performance – optimistic updates / skeletons
[ ] 7.4 Degradation – mensaje friendly al fallar

## 8. Deployability (3)
[ ] 8.1 Zero downtime – Blue/Green o Canary
[ ] 8.2 Backward compatibility – DB changes desacoplados de code
[ ] 8.3 Rollback – automatizado tras health check failure

## Resultado
Score: __/29
Decisión: PASS ( $\geq 25$ ) · CONCERNS (20–24) · FAIL ( $< 20$ )
Plan de remediación: <si CONCERNS/FAIL>
```